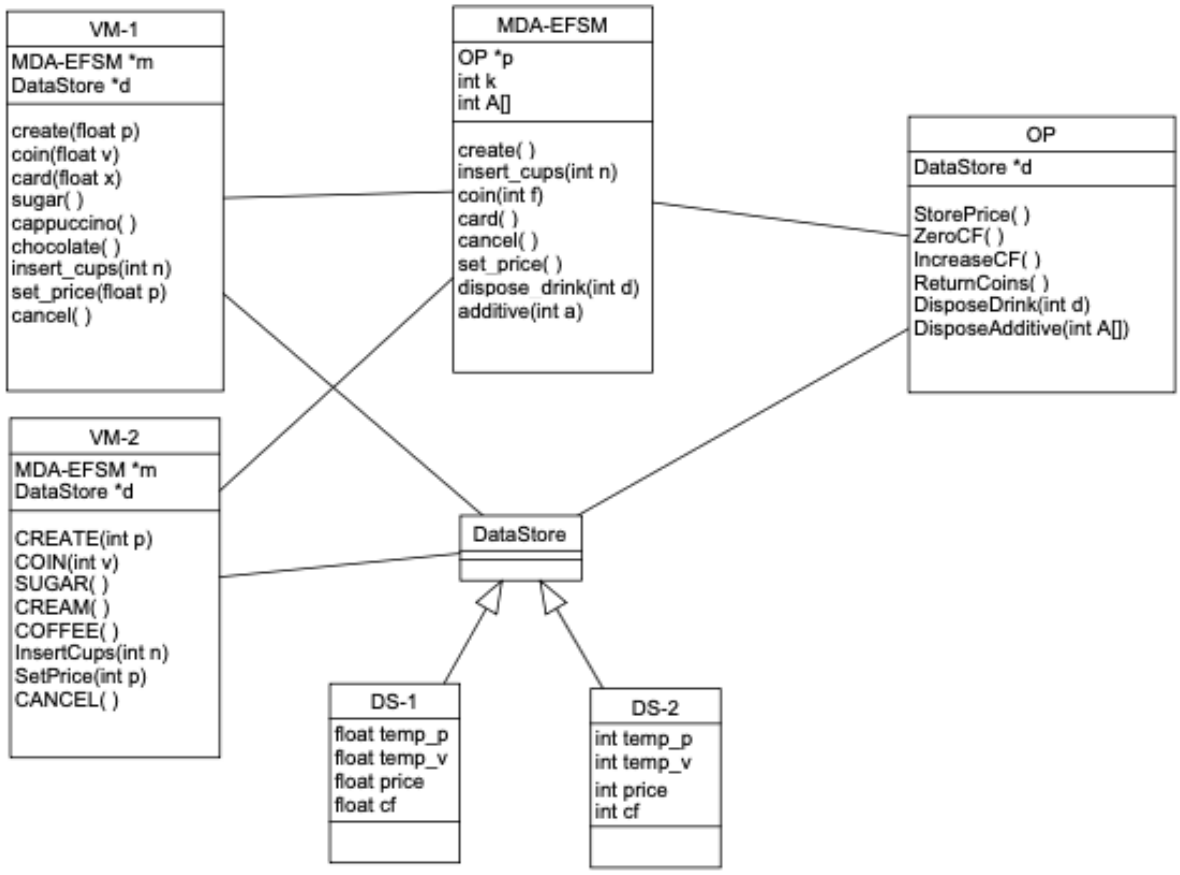
CS 586 – Project Report

Henrique Magalhaes Rio

Part 1



MDA-EFSM Events:

1. create()

2. insert_cups(int n) // *n* represents # of cups

3. coin(int f) // f=1: sufficient funds inserted for a drink, f=0: not sufficient funds for a drink

4. card()

5. cancel()

6. set_price()

7. dispose_drink(int d) // *d* represents a drink *id*

8. additive(int a) // *a* represents additive *id*

MDA-EFSM Actions:

1. StorePrice()

2. ZeroCF() // zero Cumulative Fund *cf*

3. IncreaseCF() // increase Cumulative Fund *cf*

4. ReturnCoins() // return coins inserted for a drink

5. DisposeDrink(int d) // dispose a drink with *d* id

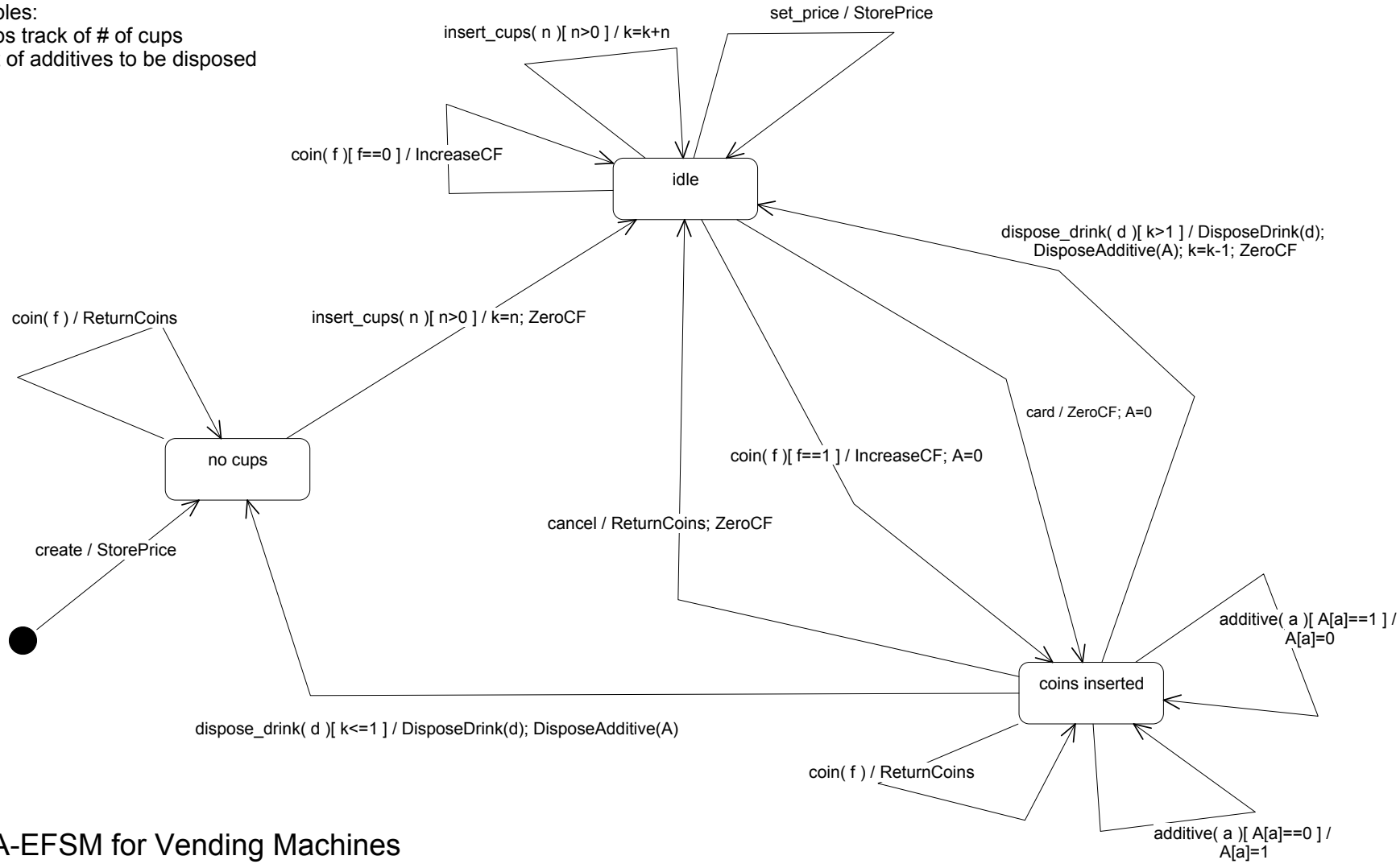
6. DisposeAdditive(int A[]) //dispose marked additives in *A* list,

// where additive with *i* id is disposed when A[i]=1

Vending-Machine-1
create(float p) {
d->temp_p=p;
m->create();
}
 	
coin(float v) {
d->temp_v=v;
if (d->cf+v>=d->price) m->coin(1);
else m->coin(0);
}
 	
card(float x) {
if (x>=d->price) m->card();
}
sugar() {
m->additive(1);
}
 	
cappuccino() {
m->dispose_drink(1);
}
 	
chocolate() {
m->dispose_drink(2);
}
 	
insert_cups(int n) {
m->insert_cups(n);
}
 	
set_price(float p) {
d->temp_p=p;
m->set_price()
}
 	
cancel() {
m->cancel();
}
	where,
	m: pointer to the MDA-EFSM
	d: pointer to the data store DS-1
	
	In the data store:
	cf: represents a cumulative fund
	price: represents a price for a drink

Vending-Machine-2
CREATE(int p) {
d->temp_p=p;
m->create();
}
 	
COIN(int v) {
d->temp_v=v;
if (d->cf+v>=d->price) m->coin(1);
else m->coin(0);
}
 	
SUGAR() {
m->additive(2);
}
 	
CREAM() {
m->additive(1);
}
 	
COFFEE() {
m->dispose_drink(1);
}
 	
InsertCups(int n) {
m->insert_cups(n);
}
 	
SetPrice(int p) {
d->temp_p=p;
m->set_price()
}
 	
CANCEL() {
m->cancel();
}
	where,
	m: pointer to the MDA-EFSM
	d: pointer to the data store DS-2
	
	In the data store:
	cf: represents a cumulative fund
	price: represents a price for a drink

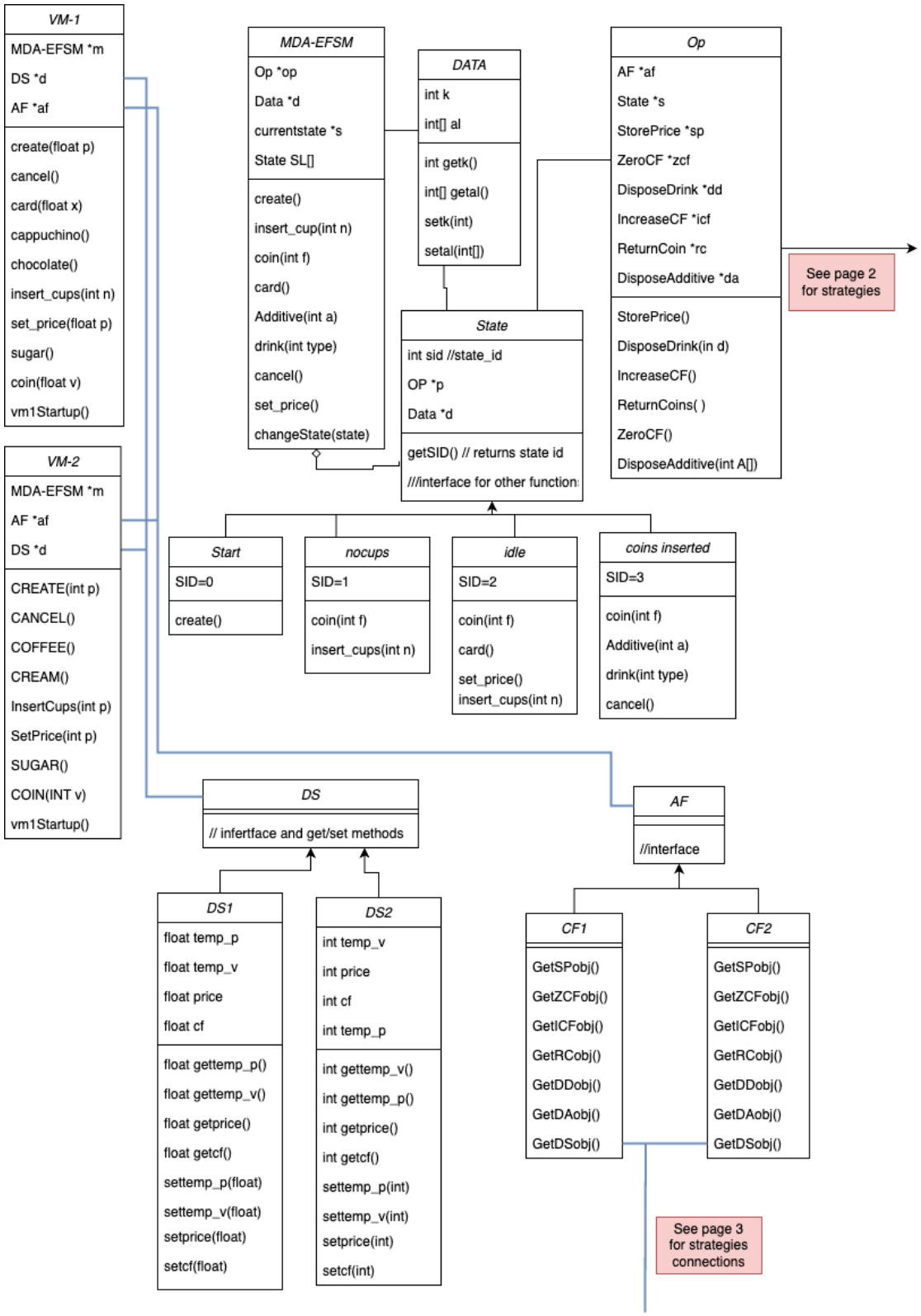
Internal Variables:
int k // keeps track of # of cups
int A[] // a list of additives to be disposed



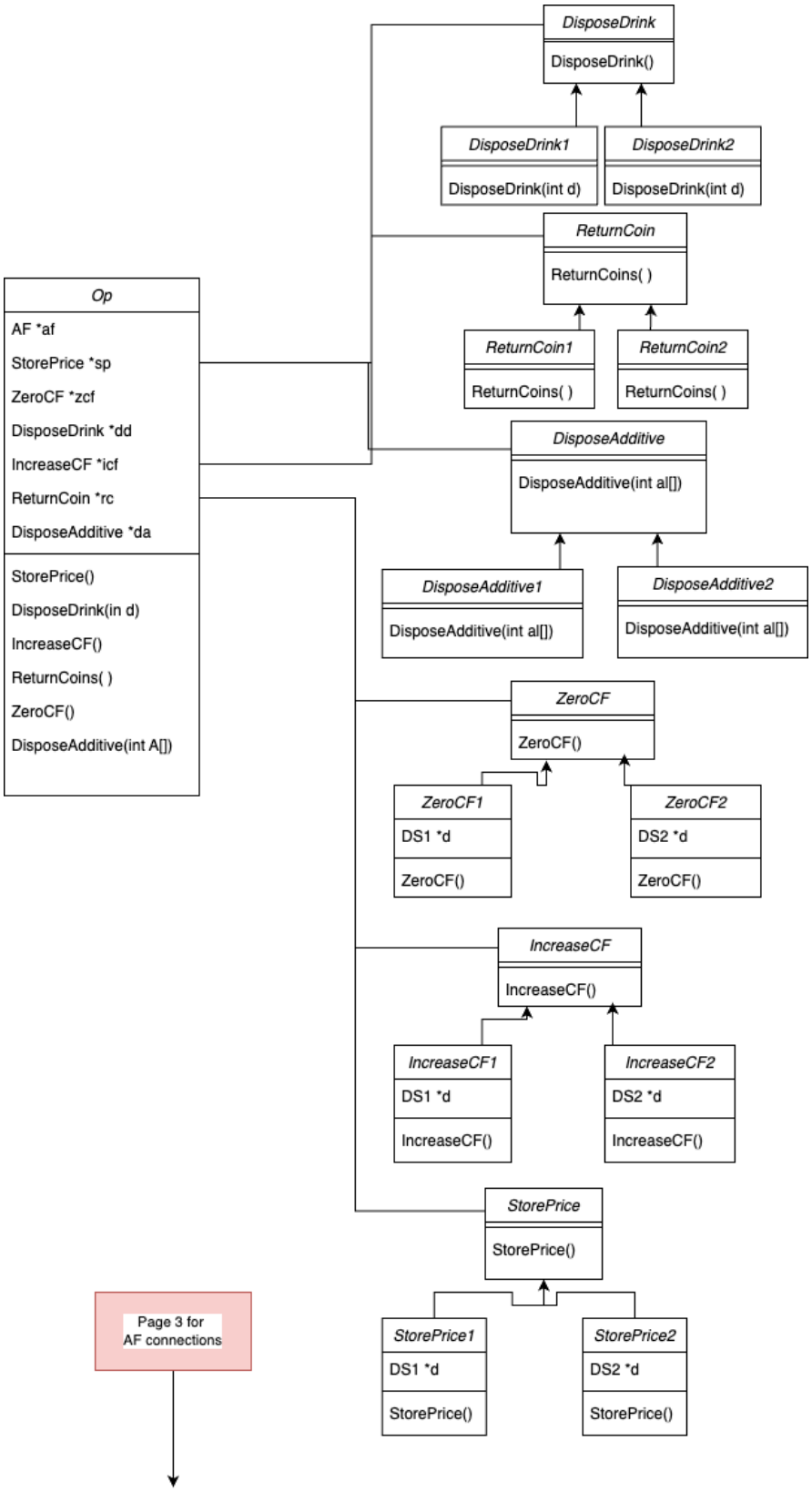
Sample MDA-EFSM for Vending Machines

Part 2

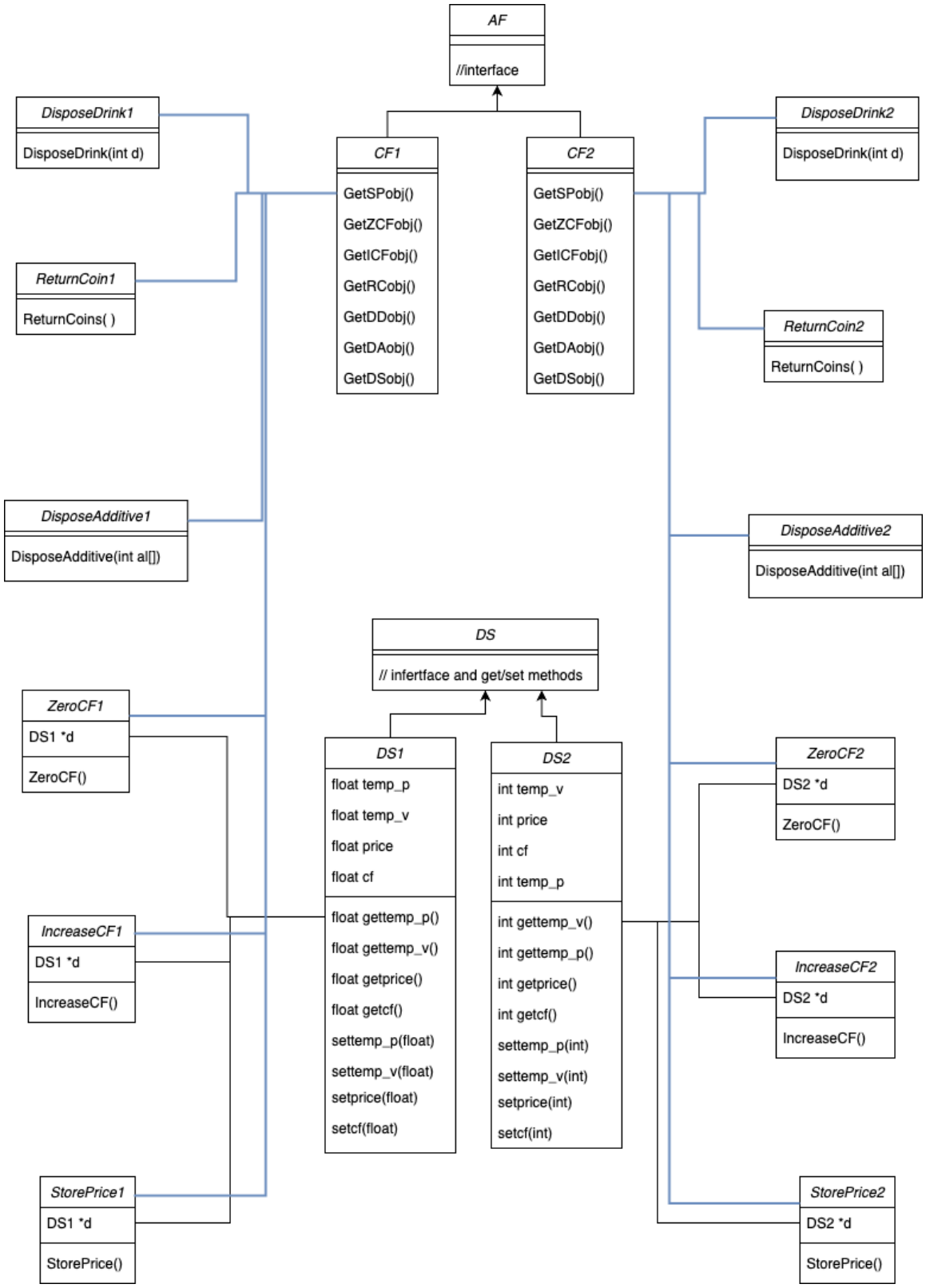
Class Diagram – Part 1



Class Diagram – Part 2



Class Diagram – Part 3



Part 3

Class – AF

Provides interface for concrete factories to implement

Functions:

getDDobj - Just the signarture of functions

getSPobj

getICFobj

getRCobj

getDAddobj

getZCFobj

getDSobj

Class – CF1

Implements the abstract factory interface that creates the necessary objects for VM-1

Functions:

getDDobj - Return a new reference to a DisposeDrink1 Object

getSPobj - Return a new reference to a StorePrice1 Object

getICFobj - Return a new reference to a IncreaseCF1 Object

getRCobj – Return a new reference to a ReturnCoin1 Object

getDAddobj – Return a new reference to a DisposeAdditive1 Object

getZCFobj – Return a new reference to a ZeroCF1 Object

getDSobj – Return a new reference to a DS1 object

Class – CF2

Implements the abstract factory interface that creates the necessary objects for VM-2

Functions:

getDDobj - Return a new reference to a DisposeDrink2 Object

getSPobj - Return a new reference to a StorePrice2 Object

getICFobj - Return a new reference to a IncreaseCF2 Object

getRCobj – Return a new reference to a ReturnCoin2 Object

getDAddobj – Return a new reference to a DisposeAdditive2 Object

getZCFobj – Return a new reference to a ZeroCF2 Object

getDSobj – Return a new reference to a DS2 object

Class – VM1

Implementation of the Vending Machine 1 classes which is where the client (driver) interaction happens.

Functions:

vm1Startup – called by the driver once VM-1 is selected, creates and connects the necessary objects using the concrete factory 1 methods.

Create – calls the method on mdaefsm and stores temporary value on ds1

Coin – calls the method on mdaefsm and and stores temporary value on ds1

Card - calls the method on mdaefsm

Sugar - calls the method on mdaefsm

Cappuccino - calls the method on mdaefsm

Chocolate - calls the method on mdaefsm

insert_cups - calls the method on mdaefsm

set_price - calls the method on mdaefsm and stores temporary value on ds1

cancel - calls the method on mdaefsm

Class – VM2

Implementation of the Vending Machine 2 classes which is where the client (driver) interaction happens.

Functions:

vm1Startup – called by the driver once VM-2 is selected, creates and connects the necessary objects using the concrete factory 2 methods.

Create – calls the method on mdaefsm and stores temporary value on ds2

Coin – calls the method on mdaefsm and and stores temporary value on ds2

Card - calls the method on mdaefsm

Sugar - calls the method on mdaefsm

Cappuccino - calls the method on mdaefsm

Chocolate - calls the method on mdaefsm

insert_cups - calls the method on mdaefsm

set_price - calls the method on mdaefsm and stores temporary value on ds2

cancel - calls the method on mdaefsm

Class – MDAEFSM

Implementation of the of the MDA-EFSM, responsible for handling the meta events and keeping track of the current state and changing the states.

Functions:

MDAEFSM (constructor) – sets up the connections to op and the states, assigns them to an array.

changeState – changes the state to the input array.

Create – perform action on the current state and changes state

insert_cup - perform action current state and changes state if necessary

coin – perform action current state and changes state if necessary

card - perform action current state and changes state if necessary

Additive - perform action current state

Drink - perform action current state and change state.

Cancel - perform action current state and change state.

set_price - perform action current state.

Class – State

Class that provides the implementation of the methods used by the state classes, also provides the references for the data and op connections.

Functions:

State (constructor) – Sets ups the references to the data and op class.

getStateId (abstract) – abstract class to return state id

create – function declaration

insert_cups - function declaration

coin – function declaration

card – function declaration

cancel – function declaration

set_price – function declaration

dispose_drink – function declaration

additive– function declaration

Class – Start

Extends the state class, and represents that start state that vending machines are initialized in, provides implementation of the necessary state classes.

Functions:

Start (constructor) – set the references to op and data using the state constructor

Create – call store price on OP

getStateId – returns 0

Class – NoCups

Extends the state class, and represents the no cups state of a vending machine , provides implementation of the necessary state classes.

Functions:

nocups (constructor) - set the references to op and data using the state constructor

coin – calls returns coins on op

insert_cups – store the amount of cups inserted on the data class.

getStateId – returns 1

Class – Idle

Extends the state class, and represents the idle state of a vending machine , provides implementation of the necessary state classes.

Functions:

idle (constructor) - set the references to op and data using the state constructor

coin – call increase current funds on op and reset the additives array

card – reset the additives array and calls zerocf on op.

set_price – call store price on op.

getStateId – returns 2

Class – coininserted

Extends the state class and represents the select drink / payment made state of a vending machine , provides implementation of the necessary state classes.

Functions:

coininserted (constructor) - set the references to op and data using the state constructor

dispose_drink – calls dispose drink on op, grabs the stored additive array from data and uses it to call dispose additive on op, calls zerocf on op. Reduces the number of cups stored in data by one.

Coin – calls returns coins on op.

Cancel - calls returns coins and zero cf on op.

getStateId – returns 3

Class – DS

Provides a generic implementation of for the data stores to implement based on whether they need integers or floats stored.

Functions:

DS (constructor) – simple constructor to initialize the variables stored in DS

getTemp_p – return temp_p

getTemp_v - return temp_v

getPrice – return price

getCf – return current fund

setTemp_p – set temporary price

setTemp_v – set temporary value

setPrice – set price

setCf – set current fund

Class – DS1

Provides float implementation of the generic DS class to be used by the vending machine 1. Does not make any changes to the methods.

Functions:

DS1 (constructor) – calls the constructor of the super class using floats instead of generics

Class – DS2

Provides integer implementation of the generic DS class to be used by the vending machine 2. Does not make any changes to the methods.

Functions:

DS1 (constructor) – calls the constructor of the super class using integer instead of generics

Class – OP

Ouput processor class used by the state classes to perform meta-actions, keeps the references for all strategy classes to be called based on the appropriate meta event.

Functions:

Op (constructor) – used to initialize all the references to the appropriate strategy based on the vending machine type

DisposeAdditive – call dispose additive on the strategy that's has been initialized

DisposeDrink - call dispose drink on the strategy that's has been initialized

StorePrice - call store price on the strategy that's has been initialized

ZeroCF - call zeroCf on the strategy that's has been initialized

IncreaseCF - call increase current fund on the strategy that's has been initialized

ReturnCoins - call return coins on the strategy that's has been initialized

Class – Data

Used to store the number of cups and the array of additional ingredients.

Functions:

Getk – returns current value of k

getAl – returns current value of al

setK – set a new value for k

setAl – set a new value for al

Class – StorePrice

Interface for StorePrice strategies to implement.

Functions:

storePrice – function declaration to be implemented by store price strategies

Class – StorePrice1

Implements store price strategy

Functions:

StorePrice 1 (constructor) - used to initialize the reference to DS1

storePrice – grabs the temp_p and stores in price.

Class – StorePrice2

Implements store price strategy

Functions:

StorePrice1 (constructor) - used to initialize the reference to DS2

storePrice – grabs the temp_p and stores in price.

Class – ZeroCF

Interface for ZeroCF strategies to implement.

Functions:

zeroCF – function declaration to be implemented by zerocf strategies

Class – ZeroCF1

Implements **ZeroCF** strategy

Functions:

ZeroCF1 (constructor) - used to initialize the reference to DS1

zeroCF– sets current funds to 0

Class – ZeroCF2

Implements **ZeroCF** strategy

Functions:

ZeroCF2 (constructor) - used to initialize the reference to DS2

zeroCF– sets current funds to 0

Class – IncreaseCF

Interface for IncreaseCF strategies to implement.

Functions:

increaseCF – function declaration to be implemented by zerocf strategies

Class – IncreaseCF1

Implements **IncreaseCF** strategy

Functions:

IncreaseCF1 (constructor) - used to initialize the reference to DS1

increaseCF– grabs the temporary value stored in DS1 and adds it to the current fund, stores it back at DS1 at cf value.

Class – IncreaseCF2

Implements **IncreaseCF** strategy

Functions:

incraseCF2 (constructor) - used to initialize the reference to DS2

increaseCF– grabs the temporary value stored in DS2 and adds it to the current fund, stores it back at DS2 at cf value.

Class – ReturnCoin

Interface for ReturnCoin strategies to implement.

Functions:

returnCoin – function declaration for Returncoin strategies to implement

Class – ReturnCoin1

Implements ReturnCoin strategies

Functions:

returnCoin – return coins

Class – ReturnCoin2

Implements ReturnCoin strategies

Functions:

returnCoin – return coins

Class – DisposeAdditive

Interface for **DisposeAdditive** strategies to implement.

Functions:

DisposeAdditive – function declaration for **DisposeAdditive** strategies to implement

Class – DisposeAdditive1

Implements **DisposeAdditive** strategies

Functions:

DisposeAdditive – disposes sugar based on the input array al

Class – DisposeAdditive2

Implements **DisposeAdditive** strategies

Functions:

DisposeAdditive – disposes sugar,cream or both based on the input array al

Class – DisposeDrink

Interface for **DisposeDrink** strategies to implement.

Functions:

DisposeDrink – function declaration for **DisposeDrink** strategies to implement

Class – DisposeDrink1

Implements **DisposeDrink** strategies

Functions:

DisposeDrink – disposes cappuccino or chocolate based on the integer passed.

Class – DisposeDrink2

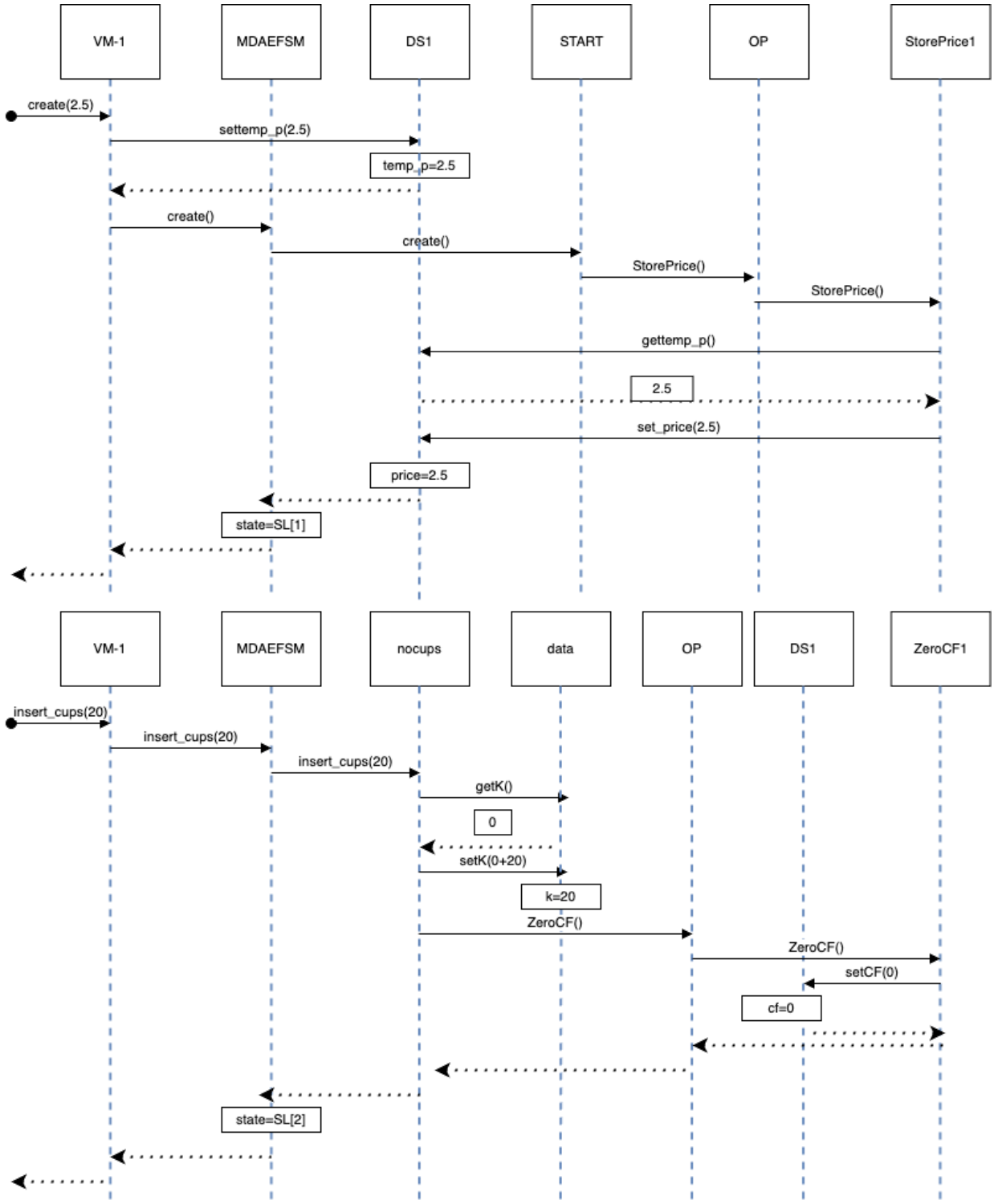
Implements **DisposeDrink** strategies

Functions:

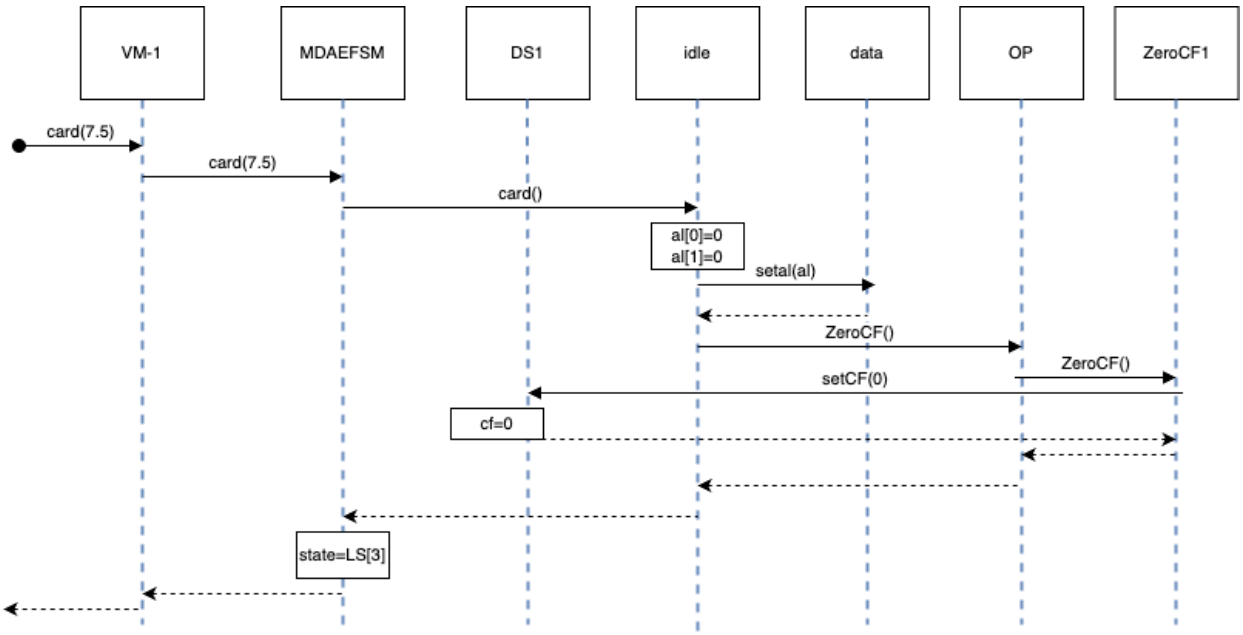
DisposeDrink – disposes Coffee.

Part 4

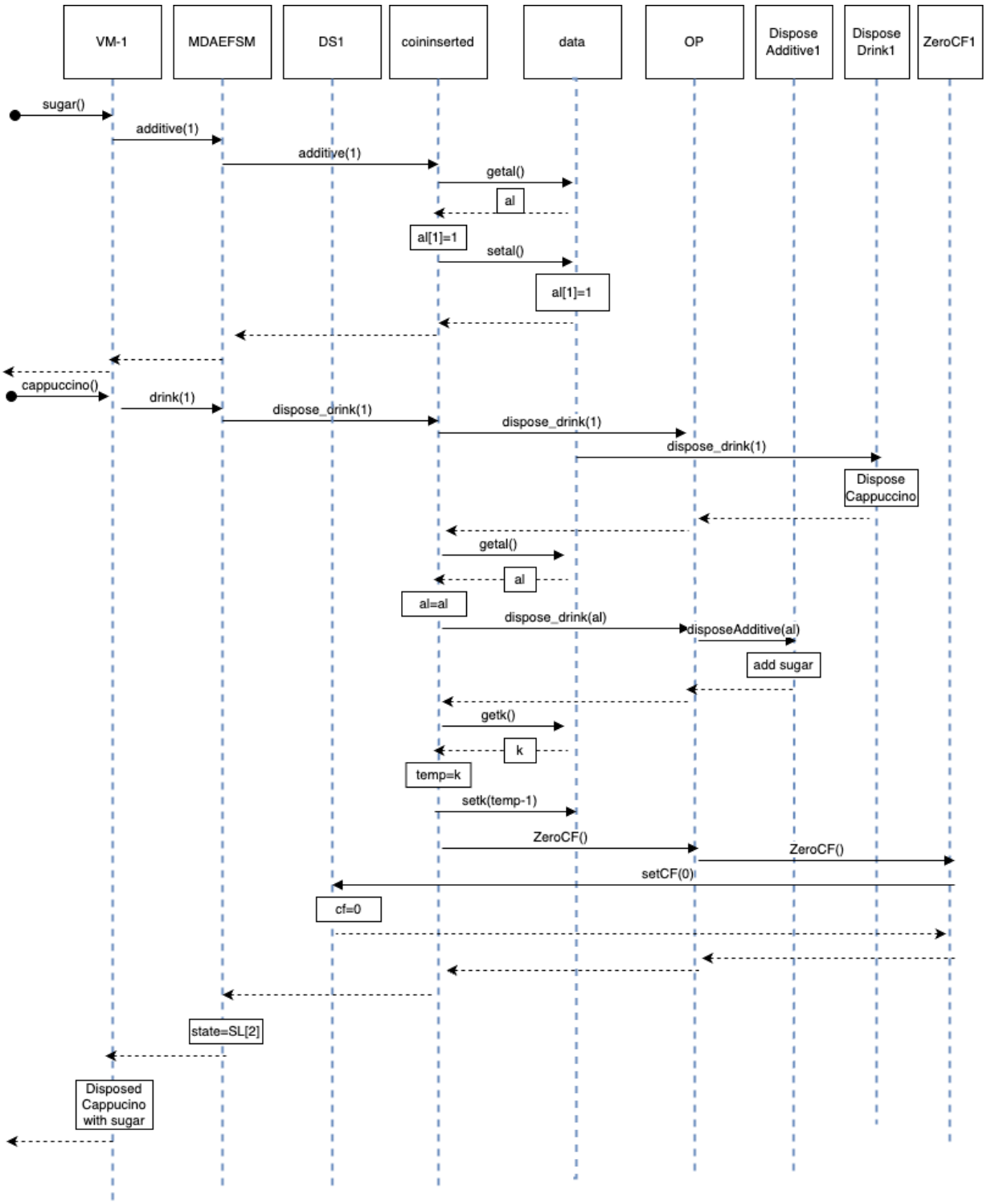
Sequence Diagram A - Part 1



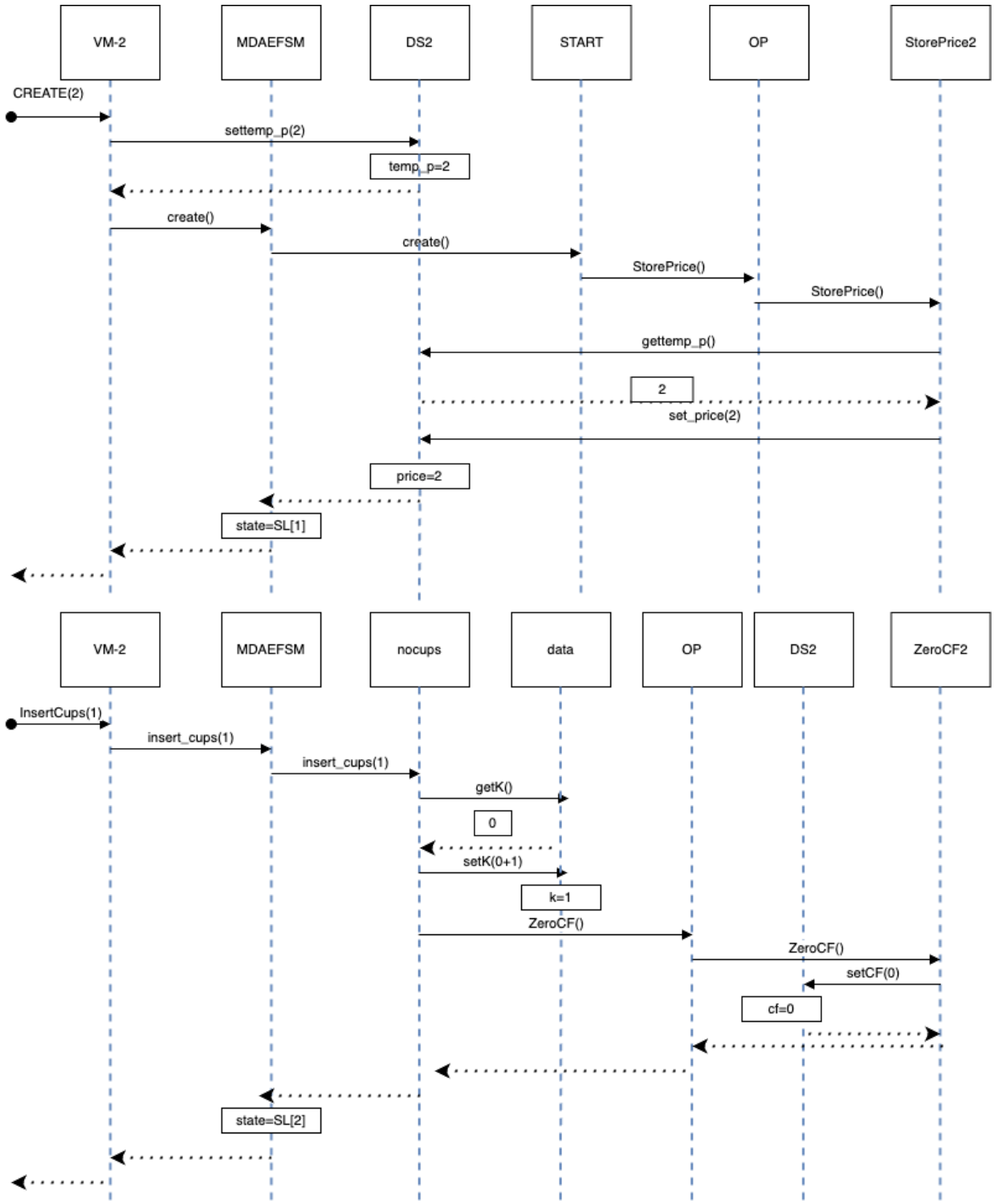
Sequence Diagram A - Part 2



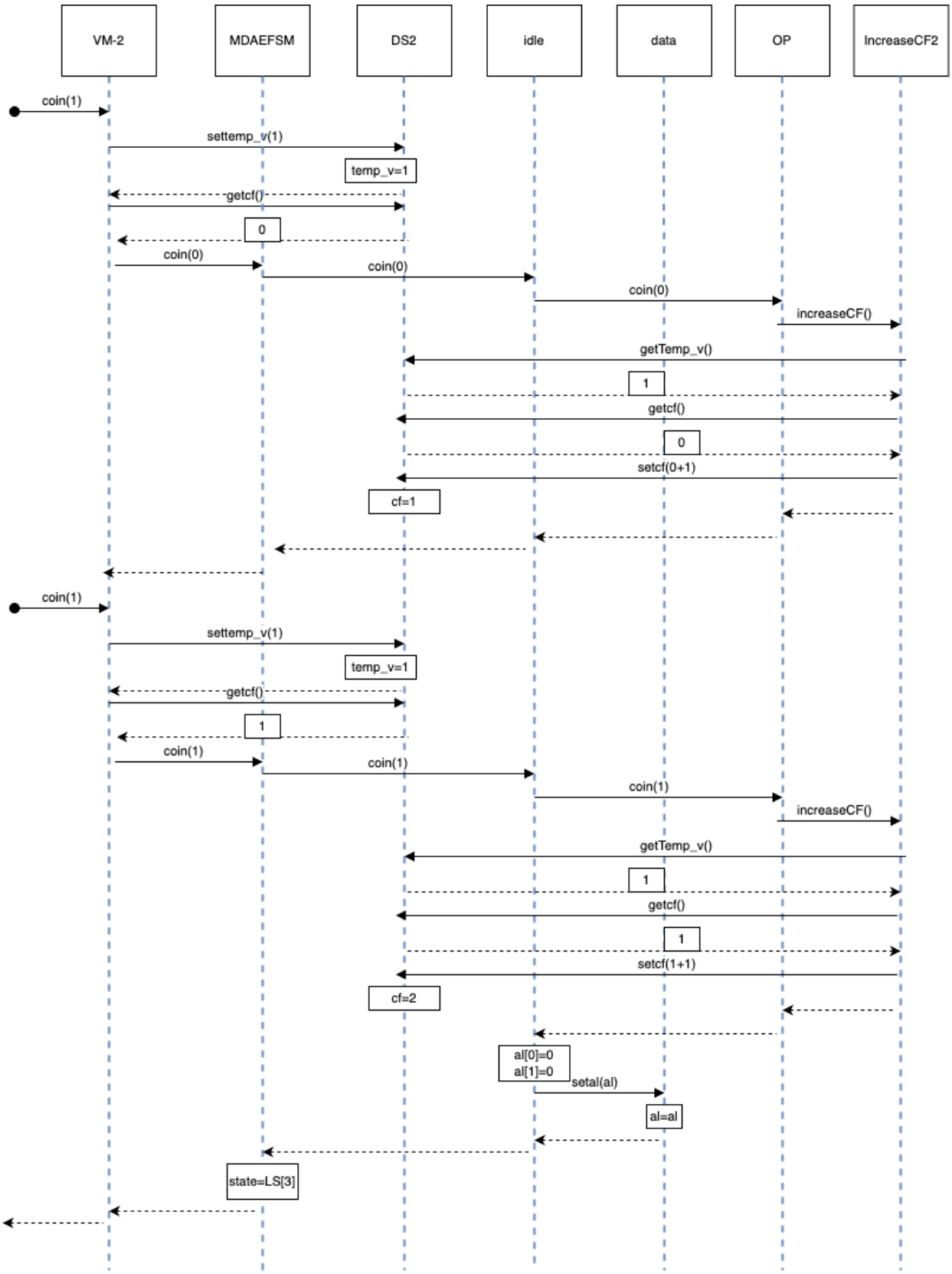
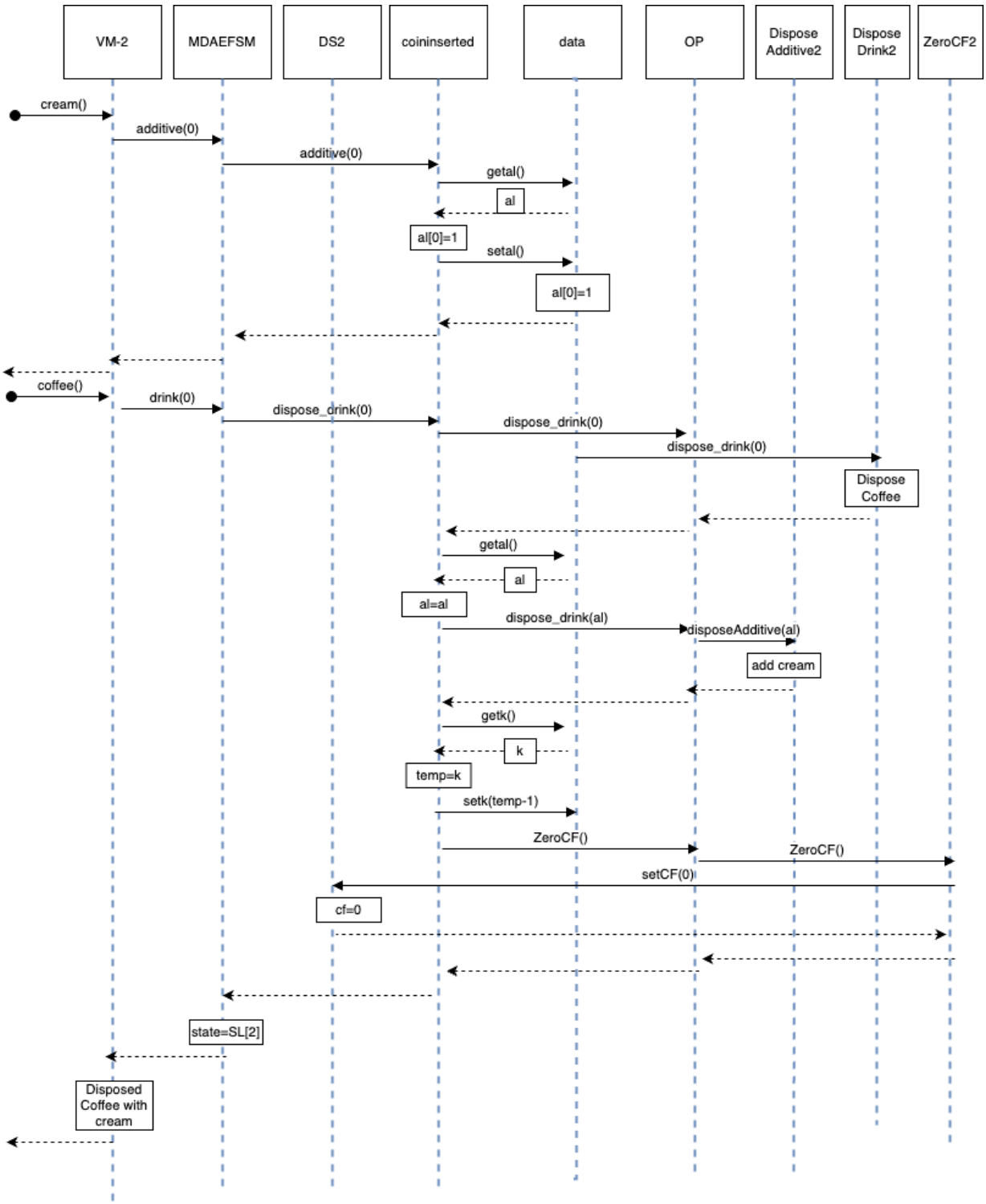
Sequence Diagram A - Part 3



Sequence Diagram B - Part 1



Sequence Diagram B - Part 2



Sequence Diagram B - Part 3